Overview of Quantum Computing

Khang Bui, Mike Lavin, Ellen Pataky, Danny Ramirez

April 2024

1 Quantum Computing

Quantum computing is the field that incorporates mathematics, physics, and computer science. Utilizing quantum mechanics, these quantum computers solve problems notably faster than classical computers.

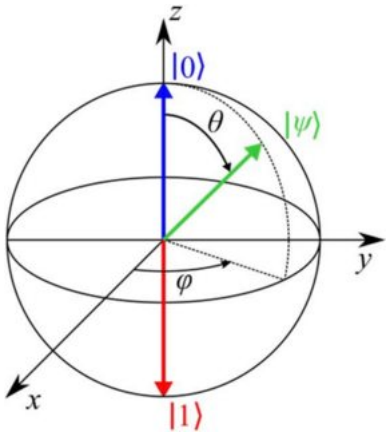
2 Why Quantum Computing?

Quantum computing makes up beautiful math. Consider bra-ket notation, linear algebra, probability, and number theory. Quantum algorithms use topics discussed in linear algebra. They use the properties of matrices and their relationships to one another, Fourier transformations, and eigenvectors. We should also study quantum computing for the sole purpose of quantum algorithms. They can solve prime number factorization problems, discrete logarithm problems, are a basis of cryptography in classical computers, and use a system of linear equations. There is so much opportunity for research in this field. This could include writing proofs, designing new algorithms, and designing new technology. There have been many breakthroughs in the field that make it fascinating. In October 2019, for Google and NASA, computing took 200 sec on a quantum computer. This would have taken 10,000 years on classical supercomputer. In December of 2020, a Chinese group used a photon-based quantum computer for a Boson sampling problem. It took 20 sec when it would have taken 600 million years on classical supercomputer. The field is growing and fascinating physicists, mathematicians, and computer scientists across the world.

3 The Qubit

The most fundamental difference between conventional computers and quantum computers is the qubit. While conventional computers use a classical bit, a quantum computer uses a quantum qubit to compute. Most people understand the classical bit and how it can either be in a state of 1 or 0. Those 1s and 0s can be strung together to represent a multitude of ideas.

On the other hand, a qubit is represented by a sphere commonly known as the Bloch Sphere. There is typically an arrow that can point in any direction around the sphere. If the arrow points exactly up, then the qubit will ultimately represent $\begin{bmatrix}1\\0\end{bmatrix}$ or $|0\rangle$. If the arrow points exactly down, then the qubit will ultimately represent $\begin{bmatrix}0\\1\end{bmatrix}$ or $|1\rangle$. This explains how a qubit can ultimately represent a classical bit, but what happens when the qubit is in any of the other states that is not $|0\rangle$ or $|1\rangle$?



[image of a Bloch Sphere]
https://quantumcomputing.stackexchange.com/questions/7161/how-are-measurements-on-z-and-x-axes-interpreted-in-the-bloch-sphere

4 Quantum Gate

In the introduction, it was discussed how classical computers use bits. These bits are constantly changed when a computer is supposed to compute something like multiplying two numbers together for example. Classical logical gates are used to change the bits in a classical computer. In quantum computers, this idea is essentially the same, but instead quantum gates are applied to qubits to change their state depending on the type of gate applied. In order to run a quantum algorithm, quantum gates are used. The following are examples of quantum gates which can be represented as unitary matrices [springer hughes].

(Feynman, Richard P. (1986). "Quantum mechanical computers". Foundations of Physics.)
From the bra ket notation section, we can see that we can represent the a a single qubit as

$$|a\rangle = v_0 |0\rangle + v_1 |1\rangle \rightarrow \begin{bmatrix}v_0\\v_1\end{bmatrix}.$$

4.1 Pauli (X/Y/Z) Gate

(Nielsen, Michael A.; Chuang, Isaac L. (2000). Quantum Computation and Quantum Information. Cambridge; New York: Cambridge University Press. ISBN 978-0-521-63235-5. OCLC 43641333)
A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. Smolin, and H. Weinfurter. Elementary gates for quantum computation. Phys. Rev. A, 52:3457–3467, 1995.
The Pauli X, Y and Z equate, respectively, to a rotation around the x, y and z axes of the Bloch sphere by π. We want to define the matrix representing these vectors and give some example of how it works. (I assume we already introduce qubit state and Bloch sphere representation)

Pauli X Gate

It is equivalent to the classical NOT gate in digital logic, which flips the state of a qubit.

$$X = \sigma_x = \text{NOT} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},$$

Multiplying the Pauli-X gate by $\vec{0}$:

$$X\vec{0} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \vec{1}$$

Multiplying the Pauli-X gate by $\vec{1}$:

$$X\vec{1} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \vec{0}$$

Pauli Y Gate

The Pauli-Y gate is a quantum gate that rotates around the Y-axis of the Bloch sphere by π radians. It performs a bit flip followed by a phase shift on the state vector.
It is represented by the matrix:

$$Y = \sigma_y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix},$$

where i is the imaginary unit. Multiplying the Pauli-Y gate by $\vec{0}$:

$$Y\vec{0} = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -i \\ 0 \end{bmatrix} = -i\vec{1}$$

Multiplying the Pauli-Y gate by $\vec{1}$:

$$Y\vec{1} = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ i \end{bmatrix} = i\vec{0}$$

Pauli Z Gate

The Pauli-Z gate is a quantum gate that performs a rotation around the Z-axis. It changes the phase of the qubit state without changing its amplitude. The Pauli-Z gate is represented by the matrix:

$$Z = \sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix},$$

Multiplying the Pauli-Z gate by $\vec{0}$:

$$Z\vec{0} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \vec{0}$$

Multiplying the Pauli-Z gate by $\vec{1}$:

$$Z\vec{1} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} = -\vec{1}$$

4.2 Hadamard Gate

Hadamard Matrix

(Source: Sylvester, J.J. (1867), Comptes Rendus de l'Académie des Sciences.)
A Hadamard matrix, fundamental in various areas of mathematics and engineering, is a square matrix whose entries are either $+1$ or -1 and whose rows are mutually orthogonal. This implies that the matrix H satisfies the condition $HH^T = nI$, where I is the identity matrix, and n is the order of H.

- **Construction:** Hadamard matrices can be constructed using the Sylvester's construction method, where an $n \times n$ Hadamard matrix is built recursively.
- **Orthogonality:** The rows (and columns) of a Hadamard matrix are orthogonal, meaning the dot product of any two distinct rows (or columns) is zero.
- **Maximal Determinant:** For any $n \times n$ matrix with entries ± 1, the determinant is at most $n^{n/2}$, with equality if and only if the matrix is a Hadamard matrix.

A key property of Hadamard matrices is their role in error correction and data compression, illustrating their practical significance beyond theoretical interest.

Example

(Paley, R.E.A.C. (1933), Journal of Mathematics and Physics.)
A simple example of a Hadamard matrix of order 4, derived using Sylvester's construction, is:

$$H_4 = \begin{bmatrix} +1 & +1 & +1 & +1 \\ +1 & -1 & +1 & -1 \\ +1 & +1 & -1 & -1 \\ +1 & -1 & -1 & +1 \end{bmatrix}, \tag{1}$$

demonstrating the recursive nature and orthogonality of Hadamard matrices. The pattern of ± 1 allows for a wide range of applications, from signal processing to coding theory.

Hadamard Gate

The Hadamard gate is a fundamental quantum gate that creates superposition states from basis states. It allows a single qubit to be in a combination of $|0\rangle$ and $|1\rangle$ states simultaneously, which is a critical feature for quantum computing. The Hadamard gate is represented by the matrix:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix},$$

Multiplying the Hadamard gate by $\vec{0}$:

$$H\vec{0} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}}(\vec{0} + \vec{1})$$

Multiplying the Hadamard gate by $\vec{1}$:

$$H\vec{1} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{\sqrt{2}}(\vec{0} - \vec{1})$$

4.3 Phase shift gates S/T

S Gate

The S gate applies a phase shift of $\pi/2$ radians. It leaves $|0\rangle$ unchanged and maps $|1\rangle$ to $i|1\rangle$. It is a root of the Z gate.
It is represented by the matrix:

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix},$$

Multiplying the S gate by $\vec{0}$:

$$S\vec{0} = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \vec{0}$$

Multiplying the S gate by $\vec{1}$:

$$S\vec{1} = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ i \end{bmatrix} = i\vec{1}$$

T Gate

The T gate applies a phase shift of $\pi/4$ radians, known as the $\pi/8$ gate. It leaves $|0\rangle$ unchanged and maps $|1\rangle$ to $e^{i\pi/4}|1\rangle$.
It is represented by the matrix:

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix},$$

Multiplying the T gate by $\vec{0}$:

$$T\vec{0} = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \vec{0}$$

Multiplying the T gate by $\vec{1}$:

$$T\vec{1} = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ e^{i\pi/4} \end{bmatrix} = e^{i\pi/4}\vec{1}$$

4.4 Controlled Gate CNOT/CZ/CCNOT

CNOT Gate

The Controlled-NOT (CNOT) gate is a two-qubit quantum gate that performs a conditional operation, flipping the second (target) qubit if the first (control) qubit is in the state $|1\rangle$. It is fundamental in quantum computing for entangling qubits and constructing more complex quantum algorithms.
It is represented by the matrix:

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix},$$

Multiplying the CNOT gate by $\vec{00}$ (control qubit = 0, target qubit = 0):

$$\text{CNOT}\vec{00} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \vec{00}$$

Multiplying the CNOT gate by $\vec{11}$ (control qubit = 1, target qubit = 1):

$$\text{CNOT}\vec{11} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \vec{10}$$

CZ/CCNOT (should we also write this?)

4.5 General Case Example

Let's consider a general quantum state vector ψ given by:

$$\psi = \begin{bmatrix} a \\ b \end{bmatrix}$$

Pauli X Gate

$$X\psi = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} b \\ a \end{bmatrix}$$

Pauli Y Gate

$$Y\psi = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} -ib \\ ia \end{bmatrix}$$

Pauli Z Gate

$$Z\psi = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} a \\ -b \end{bmatrix}$$

Hadamard Gate

$$H\psi = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} a+b \\ a-b \end{bmatrix}$$

S Gate

$$S\psi = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} a \\ ib \end{bmatrix}$$

T Gate

$$T\psi = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} a \\ e^{i\pi/4}b \end{bmatrix}$$

CNOT Gate

Multiplying the CNOT gate by a general two-qubit state $[a, b, c, d]$:

$$\text{CNOT}\psi = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} a \\ b \\ d \\ c \end{bmatrix}$$

5 Linear Algebra in Quantum Computing

Source: https://learn.microsoft.com/en-us/azure/quantum/overview-algebra-for-quantum-computing

A qubit's state is depicted by a column vector in a two-dimensional complex vector space. This vector is usually represented as:

$$|\psi\rangle = \begin{pmatrix} a \\ b \end{pmatrix}$$

Here, a and b are complex numbers that satisfy the normalization condition $|a|^2 + |b|^2 = 1$, ensuring that the probabilities of the qubit being in states 0 and 1 sum up to 1.

Quantum operations are symbolized by unitary matrices. A matrix U is unitary if $U^\dagger U = I$, where $U^\dagger$ represents the conjugate transpose of U and I is the identity matrix.

For instance, the Pauli-X gate (or NOT gate) is represented by the matrix:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Applying the Pauli-X gate to a qubit state $|\psi\rangle$ means multiplying the state vector by the matrix X:

$$X|\psi\rangle = X \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} b \\ a \end{pmatrix}$$

Similarly, the Hadamard gate H is represented by:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Applying the Hadamard gate to a qubit state $|\psi\rangle$ yields:

$$H|\psi\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} a+b \\ a-b \end{pmatrix}$$

When describing the joint state of two qubits, we utilize the tensor product operation. For instance, let's consider two single-qubit states represented by the following column vectors:

$$|\psi\rangle = \begin{pmatrix} a_1 \\ b_1 \end{pmatrix} \quad \text{and} \quad |\phi\rangle = \begin{pmatrix} a_2 \\ b_2 \end{pmatrix}$$

The tensor product of these states, denoted $|\psi\rangle \otimes |\phi\rangle$, is calculated as:

$$|\psi\rangle \otimes |\phi\rangle = \begin{pmatrix} a_1 \\ b_1 \end{pmatrix} \otimes \begin{pmatrix} a_2 \\ b_2 \end{pmatrix} = \begin{pmatrix} a_1 \begin{pmatrix} a_2 \\ b_2 \end{pmatrix} \\ b_1 \begin{pmatrix} a_2 \\ b_2 \end{pmatrix} \end{pmatrix} = \begin{pmatrix} a_1 a_2 \\ a_1 b_2 \\ b_1 a_2 \\ b_1 b_2 \end{pmatrix}$$

Here, $|\psi\rangle$ and $|\phi\rangle$ are represented by $\begin{pmatrix} a_1 \\ b_1 \end{pmatrix}$ and $\begin{pmatrix} a_2 \\ b_2 \end{pmatrix}$ respectively. The resulting four-dimensional column vector indicates the probabalistic state of the two qubits after applying the tensor product operation.

6 Superposition, Entanglement, Interference

6.1 Superposition

A qubit is represented as a 3-dimension space with an arrow. A "0" is represented by an arrow pointing up, and a "1" is represented by a down pointing arrow. When an arrow points in a different direction, not representing a 0 or 1, it is superposition. Wherever it is pointing is the probability that the qubit will represent a 0 or 1.

6.2 Entanglement

Furthermore, two or more qubits can be tied together in entanglement. Entanglement means that instead of one qubit being evaluated at a time, multiple qubits are 'tied' together and analyzed as a whole. Thus for only two qubits, there are four possible states: 00, 01, 10, or 11. Then there is a probability associated with each possible state for how likely it is for the entangled qubits to fall to a single state. This is the heart and soul as to why quantum computers are so fast.

6.3 Interference

A third property important to know is the notion of interference. Quantum particles act as waves, so a combination of quantum particles can lead to destructive and constructive interference. Destructive interference occurs when there are opposing waves such that one wave is at its peak, the other is at the bottom of the trough, and they have similar frequency. The combination of these two waves would cancel each other out and result in an overall wave that is flat. The opposite occurs when two waves match each other and create an overall amplified wave function. A way to avoid interference is by having any two waves separate from each other.

7 Bra-ket Notation

7.1 Hilbert Space

(Source: Rudin, Walter (1987), Real and Complex Analysis, McGraw-Hill, ISBN 978-0-07-100276-9)
It is good to understand the vector space that we will be working on for most of our operation, the Hilbert space. There are several different materials of Hilbert

space, but here is a version from Rudin (1987):
A Hilbert space H is a vector space over the field of real or complex numbers, equipped with an inner product $\langle \cdot, \cdot \rangle$ that satisfies the following properties:

- **Linearity:** For any vectors $x, y, z \in \mathrm{H}$ and scalar α, the inner product satisfies $\langle \alpha x + y, z \rangle = \alpha \langle x, z \rangle + \langle y, z \rangle$.
- **Conjugate Symmetry:** For any vector $x, y \in \mathrm{H}$, $\langle x, y \rangle = \overline{\langle y, x \rangle}$, where the bar denotes the conjugate complex.
- **Positivity:** For any vector $x \in \mathrm{H}$, $\langle x, x \rangle \geq 0$, with equality if and only if $x = 0$.

H must be complete with respect to the norm induced by the inner product, meaning that every Cauchy sequence in H converges to an element in H.

7.1.1 Example

(Dieudonné, Jean (1960), Foundations of Modern Analysis, Academic Press.)
One basic example of Hilbert space is the sequence space, l^2. This space includes all infinite sequences of real (or complex) numbers $(a_1, a_2, a_3, \ldots)$ for which the sum of the squares of the elements is finite, i.e.,

$$\sum_{n=1}^{\infty} |a_n|^2. < \infty \tag{2}$$

The inner product in l^2 is defined as

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{n=1}^{\infty} a_n \overline{b_n}, \tag{3}$$

for any two sequences $\mathbf{a} = (a_1, a_2, a_3, \ldots)$ and $\mathbf{b} = (b_1, b_2, b_3, \ldots)$ in l^2, where $\overline{b_n}$ denotes the complex conjugate of b_n.
There are also other more complicated example of H space like Lebesgue spaces or Sobolev spaces, but we will not dig deep into this because this is just one of the basic concepts of our problem.)
(Fun fact: A pure quantum state can be represented as a vector $|\psi\rangle$ in a complex Hilbert space. However, it is more accurately described as a ray, or a one-dimensional subspace of the Hilbert space.)
The bra–ket notation is particularly useful in Hilbert spaces which have an inner product.

7.2 Bra-ket notation

Bra–ket notation was effectively established in 1939 by Paul Dirac and is thus also known as the Dirac notation. (Dirac, P. A. M. (1939). "A new notation for quantum mechanics". Mathematical Proceedings of the Cambridge

Philosophical Society. 35 (3): 416–418).
The Dirac Bra-Ket notation is a way to describe quantum states. The ket vector is written as the following symbol, $|k\rangle$, which as a column vector can be represented as the following, $\begin{pmatrix} k_1 \\ k_2 \end{pmatrix}$ where the vector is composed of complex numbers. For context, the meaning would be that a physical system is in quantum state k, where k is representing a physical quantity like momentum or spin [unlv.edu]. The ket can also be described as the initial state in some event. The kets $|0\rangle$ and $|1\rangle$ represent the two logical states of qubits. Since they are qubits, superposition can be represented as the following,

$$|k\rangle = x\,|0\rangle + y\,|1\rangle$$

where x and y are complex numbers [libretexts]. Kets are also assumed to be normalized, meaning the dot product of a ket by itself is equal to 1. This fact explains why in linear algebra, if a qubit is represented as a single column matrix, $\begin{bmatrix} \alpha \\ \beta \end{bmatrix}$, it is a requirement that $|\alpha|^2 + |\beta|^2 = 1$. This equation represents the probability of the qubit collapsing one way or another [microsoft article].

In Dirac notation there also exists a bra vector represented as, $\langle p|$. The bra can be described as the final state of some event. The bra vector of a corresponding ket vector is the Hermitian conjugate of a ket which means the bra is the transpose of a ket where all the elements of the vector have been complex conjugated [libretexts]. An example would be the following: Working in matrix form, let

$$|k\rangle = \begin{bmatrix} 4 & 2+i \\ 2-i & -6 \end{bmatrix}$$

Then the conjugate gives

$$\begin{bmatrix} 4 & 2-i \\ 2+i & -6 \end{bmatrix}$$

Finally the $\langle p|$ = the transpose which is =

$$\begin{bmatrix} 4 & 2+i \\ 2-i & -6 \end{bmatrix}.[byjus]$$

Using the ket and bra, the inner product is represented as $\langle k|p\rangle$, which is a complex number and also known as the probability amplitude [qmchanics paper].

For example, $\langle x = .45|p\rangle$ is the probability amplitude that a particle in state p will be found at position $x = .45$ [ju.edu]. Also, if we square the same equation, $|\langle x = .45|p\rangle|^2$ is is the probability density that a particle in state p will be found at position $x = .45$ [ju] or the probability of finding a particle at position $x = .45$ when a measurement is made on that particle in the state $|p\rangle$.

Also, in Dirac notation there are operators that transform one ket vector into another ket vector. An example of an operator is the outer product which is shown as, $|k\rangle\langle p|$. For example, the outer product of a vector with itself, $|k\rangle\langle k|$,

means that it projects a vector $|p\rangle$ in the Hilbert Space to the one-dimensional subspace of the Hilbert Space spanned by $|k\rangle$ [batista]. Operators, similar to ket vectors can represent the physical systems of a quantum state. For the purpose of this paper, it is not extremely important that we discuss this topic in a lot of detail.

8 Black Box Algorithms

In quantum computing, black box algorithms are a class of algorithms where the inner workings of the algorithm, or the specific details of the problem being solved, are hidden or abstracted away. Instead, these algorithms interact with the problem through a "black box" that provides input and receives output, without revealing the internal mechanisms.

Black box algorithms are essential in quantum computing because they encapsulate complex quantum operations and enable the exploitation of quantum phenomena without requiring a deep understanding of quantum mechanics.